

Matoůšek Partition Tree: Short Technical Report

Goal

This project implements one constructive step of Matoůšek's 2D Partition Theorem for halfplane range counting. The theorem says that n planar points can be split into about r triangular groups so that every query line crosses only

$$O(\sqrt{r})$$

groups. A query can count non-crossed groups wholesale and recurse only into crossed groups.

The theorem is not new here, and neither is its cutting machinery, which is due to Chazelle and Friedman. The contribution is implementation and measurement: build the construction, check the proof invariants at runtime, and measure the constants hidden in the asymptotic bound.

Principle

The construction is easiest to understand as a way to avoid checking all possible query lines.

There are infinitely many lines, so the algorithm first builds a finite test set Q . It then constructs the point groups while keeping the crossing count small for every line in Q . A separate lemma, the Test Set Lemma, transfers this finite guarantee to all query lines.

So the proof has two levels:

1. Control the finite test set:

$$K_Q = O(\sqrt{r}).$$

2. Transfer that bound to every line:

$$\text{cr}_\Pi(h) \leq 3K_Q + \sqrt{r}.$$

Algorithm Summary

The construction has three main ingredients.

Duality. Each point $p = (a, b)$ is mapped to the line $p^* : y = ax - b$. This turns reasoning about query lines into reasoning about cells in the dual plane.

Cuttings. A cutting divides the dual plane into cells that are crossed by only a controlled number of lines. A first cutting builds a finite test set Q with

$$|Q| \leq r.$$

This implementation uses cutting scale $t = \beta\sqrt{r}$ with $\beta = 0.25$, and checks the size condition directly.

Reweighting. Each test line starts with weight 1. Whenever a chosen group triangle crosses a test line, that line's weight doubles. Future weighted cuttings avoid high-weight lines. This keeps the worst test-line crossing count

$$K_Q = \max_{q \in Q} \kappa(q)$$

near a constant multiple of $\sqrt{r}$.

Correctness Checks

The code raises an error instead of silently continuing when a proof condition fails. The main runtime checks are:

- every cutting cell satisfies the crossing-weight bound;
- the number of cutting faces stays within the pigeonhole budget;
- the test set satisfies $|Q| \leq r$;
- groups are disjoint and contain the required points;
- exact query answers match brute force tests.

All geometric predicates use exact rational arithmetic through `Fractions.Fraction`. These checks are not a formal proof certificate, but they make the implementation auditable.

Experiment

The experiment uses $n = 1200$ uniformly random points with seed 7 and applies one partition step, not the full recursive tree:

```
python3 benchmarks/measure_crossings.py 1200 7.
```

Table 1: Crossing measurements for one verified partition step.

r	groups	$ Q $	K_Q	$K_Q/\sqrt{r}$	random max	$3K_Q + \sqrt{r}$
25	25	17	20	4.00	25	65 (vacuous)
36	36	31	25	4.17	34	81 (vacuous)
64	66	55	39	4.88	48	125 (vacuous)

Takeaway

The mechanism works: in these tests, K_Q grows like $\sqrt{r}$, with measured constant about 4–5.

The constants are large. The Test Set Lemma transfers the test-line bound to all lines using

$$\text{cr}_\Pi(h) \leq 3K_Q + \sqrt{r}.$$

With the measured constants, this bound is still larger than the total group count until roughly $r = 170$. At that scale, the exact rational construction is already slow.

So the project demonstrates the theorem’s mechanism clearly, but also shows why this theory-optimal construction is hard to turn into a practical data structure at moderate input sizes.

Repository

Code, tests, benchmarks, the full technical report, and the longer derivation are available at: <https://github.com/wangyi1010/matousek-partition-tree>.